

Foistlinge – Vorbestell-Tool

Landingpage zum Vorbestellen von "Hymnen für Trinker" (Vinyl, 24 € inkl. Versand) mit direkter PayPal-Zahlung, automatischer Adressabfrage durch PayPal und einem passwortgeschützten Admin-Bereich mit Verkaufszähler + Excel-Export.

Die Seite ist bereits live: <https://foistlinge-vorbestellung.onrender.com>

Für die vollständige Dokumentation (Architektur, Admin-Anleitung, Live-Zugangsdaten-Übersicht, bekannte Einschränkungen, Datenschutz-Status, Wartung) siehe **DOKUMENTATION.md**. Diese README beschreibt nur die Ersteinrichtung von Grund auf.

1. Lokal starten

```
npm install
cp .env.example .env
```

Dann `.env` ausfüllen (siehe Schritt 2) und starten:

```
npm start
```

- Landingpage: <http://localhost:3000>
- Admin-Bereich: <http://localhost:3000/admin> (Login aus `.env`)

2. PayPal-App einrichten

1. Auf <https://developer.paypal.com> einloggen (dein normaler PayPal-Business-Account reicht).
2. "Apps & Credentials" -> "Create App" -> Namen vergeben (z. B. "Foistlinge Vorbestellung").
3. Zuerst im Modus **Sandbox** testen: Client ID und Secret aus der Sandbox-App in `.env` eintragen (`PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`), `PAYPAL_ENV=sandbox`.
4. Mit einem PayPal-Sandbox-Testkäufer-Account (unter "Sandbox" -> "Accounts" im Dashboard) einmal durchbestellen und prüfen, ob die Bestellung im Admin-Bereich auftaucht.
5. Wenn alles passt: im Dashboard oben rechts auf **Live** umschalten, dort die echte Client ID/Secret kopieren, in `.env` eintragen und `PAYPAL_ENV=live` setzen. Ab jetzt wird echtes Geld abgebucht.

Das Geld landet direkt auf dem PayPal-Konto, mit dem die App erstellt wurde.

3. Admin-Zugang absichern

In `.env` `ADMIN_USER` und `ADMIN_PASS` auf eigene Werte setzen, bevor die Seite live geht.

4. Datenbank einrichten (Neon, kostenlos & dauerhaft)

1. Auf <https://neon.tech> mit GitHub- oder E-Mail-Account registrieren.
2. "Create Project" -> Namen vergeben (z. B. "foistlinge-vorbestellung") -> Region z. B. Frankfurt (eu-central-1) auswählen.
3. Im Dashboard unter "Connection Details" die **Connection String** kopieren (sieht aus wie `postgresql://user:pass@ep-xxxx.eu-central-1.aws.neon.tech/neondb?sslmode=require`).
4. Diesen String später bei Render als `DATABASE_URL` eintragen (Schritt 6).

Ohne `DATABASE_URL` läuft die App weiterhin lokal mit der einfachen `data/orders.json`-Datei – praktisch zum Testen, aber nicht für den dauerhaften Live-Betrieb gedacht.

5. Code auf GitHub bringen

1. Auf <https://github.com> kostenlos registrieren (falls noch nicht geschehen).
2. Neues, **privates** Repository anlegen (z. B. "foistlinge-vorbestellung").
3. Diesen Projektordner dorthin pushen:

```
git init
git add .
git commit -m "Vorbestell-Tool"
git branch -M main
git remote add origin https://github.com/<dein-username>/foistlinge-vorbestellung.git
git push -u origin main
```

`.env` wird durch `.gitignore` automatisch NICHT mitgeschickt – die Zugangsdaten bleiben also lokal.

6. Online stellen mit Render (damit der QR-Code am Veranstaltungstag funktioniert)

1. Auf <https://render.com> registrieren (Login z. B. direkt mit GitHub).
2. "New" -> "Web Service" -> das eben erstellte GitHub-Repo auswählen.
3. Build Command: `npm install`, Start Command: `npm start`.
4. Unter "Environment" alle Variablen eintragen (wie in `.env`): `PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`, `PAYPAL_ENV`, `DATABASE_URL`, `ADMIN_USER`, `ADMIN_PASS`, `ALBUM_PRICE`, `ALBUM_SHIPPING`, `ALBUM_CURRENCY`.
5. Deployen lassen. Nach ein paar Minuten ist die App unter einer URL wie `https://foistlinge-vorbestellung.onrender.com` erreichbar.
6. `/admin` mit den echten Zugangsdaten öffnen und testen, `/qr.png` prüfen (zeigt automatisch auf die Render-URL).

Hinweis: Der kostenlose Render-Plan schläft nach Inaktivität kurz ein – der erste Aufruf nach einer Pause dauert dann ca. 30–50 Sekunden länger. Für den Veranstaltungstag reicht das meist aus; falls nicht, gibt es bei Render einen günstigen "Always On"-Plan.

7. QR-Code für die Veranstaltung

Im Admin-Bereich (`/admin`) unter "QR-Code für die Veranstaltung anzeigen" – zeigt automatisch auf die aktuell aufgerufene Domain, also direkt nach dem Deploy einsatzbereit. Einfach als Bild speichern/ausdrucken.

Wie die Daten gespeichert werden

Jede erfolgreich bezahlte Bestellung (Name, E-Mail, Lieferadresse, Betrag, PayPal-Order-ID, Datum) landet in der Postgres-Datenbank (`DATABASE_URL`), sobald diese konfiguriert ist – sonst als Fallback in `data/orders.json` . Im Admin-Bereich gibt es zusätzlich jederzeit den Button **"Als Excel exportieren"** für eine aktuelle `.xlsx` -Datei mit allen Bestellungen.

Preis ändern

`ALBUM_PRICE` (Album ohne Versand), `ALBUM_SHIPPING` (Versandkosten) und `ALBUM_CURRENCY` in `.env` anpassen. Angezeigt und über PayPal abgebucht wird immer die Summe aus beiden (aktuell 18,00 € + 6,00 € = 24,00 €).